Can a Smaller Network Keep Its Promise?

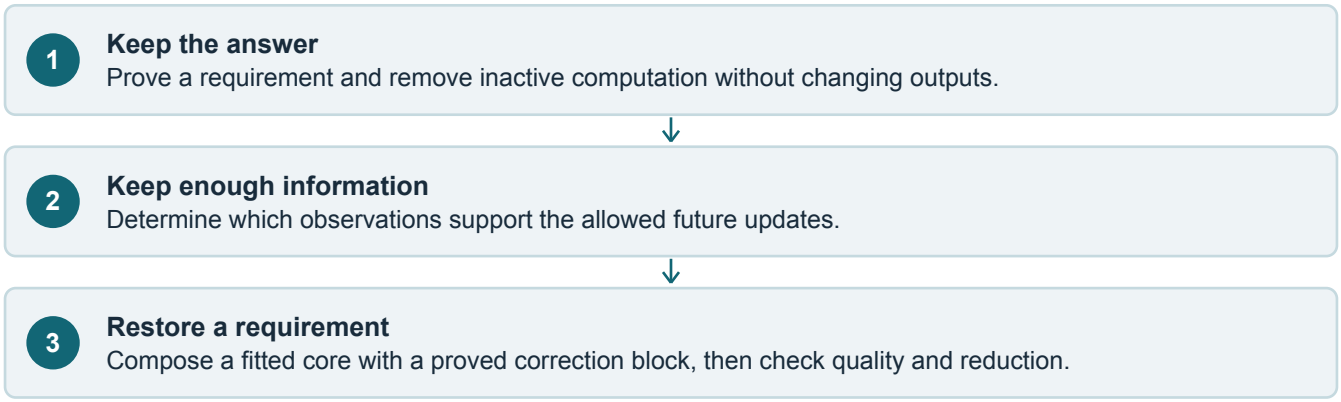
A worked path from certified reduction and stable views to controller repair

Worked guide • revised 7 November 2026

Imagine two almost identical measurements arriving at a drone's controller. We want the resulting commands to remain close. We begin by proving that precise requirement on two published networks, then make the networks smaller while preserving every output on the chosen domain. The guarantee survives because the answers have not changed.

Once we remove computation, a second question follows: **what information can we forget if the parameters will change later?** A parameter that does not affect today's output might affect tomorrow's update. This leads to a retained view of the state, an equivalence relation called RR, and a stable subalgebra describing what remains computable from that view.

Preservation also exposes a limitation: if the original answer violates a requirement, an exactly equal smaller network keeps that violation. Our next step is therefore **repair by composition**. A fitted neural core proposes an output; a proved correction block uses the input and that proposal to enforce a directional requirement. We then check reference quality and certify reduction of the fitted core again.



These questions organize the guide. Lean checks the mathematical arguments at each stage. The examples use different saved models: DL2/Gödel for the opening robustness result, and Adversarial for the view and repair studies. Each transition names what is retained, what changes and which guarantee has actually been established. Sections 10–12 bring the repair into the main story; Appendix E gives its complete mathematical workflow.

1. The promise, stated explicitly

A **checkpoint** is a saved set of learned network parameters. Here the network takes six normalized real numbers and produces one scalar command. Normalized inputs lie between 0 and 1; they are not physical-unit readings. Write **N(x)** for the network's output at input vector x.

An exact condition that is true or false for given arguments is a **predicate**. The input predicate **P** says that both inputs are in the permitted cube and that each of their six coordinates differs by at most 0.001:

$$P(x, y) \iff x, y \in [0, 1]^6 \ \wedge \ \forall i \in \{0, \ldots, 5\}, \ |x_i - y_i| \leq 0.001.$$

The output requirement **Q** says that the two commands differ by at most 0.1:

$$Q_N(x, y) \iff |N(x) - N(y)| \leq 0.1.$$

Lean proves this implication for each original and reduced DL2/Gödel network:

$$\forall x, y \in \mathbb{R}^6, \qquad P(x, y) \implies Q_N(x, y).$$

For example, take all six coordinates of x to be **0.5**, and all six coordinates of y to be **0.5005**. Their differences are 0.0005, so P holds. The theorem guarantees Q for this pair and for every other permitted pair.

Published checkpoint	Required maximum change	Proved maximum when P holds
DL2	0.1	0.031
Gödel	0.1	0.026

The names identify two archived model files. Their proofs cover all permitted input pairs, including pairs absent from the training data.

2. What does “every permitted pair” mean?

The six input coordinates represent normalized horizontal and vertical velocity, angular velocity, angle, and horizontal and vertical position. We use x and y for **whole input vectors**; a subscript such as x_4 selects one coordinate. All six coordinates can change together, independently.

Keep x fixed at (0.5, 0.5, 0.5, 0.5, 0.5, 0.5) and consider these choices:

Second input y	Does P hold?	What the displayed implication tells us
Every coordinate is 0.5005	Yes: all differences are 0.0005.	The output difference is at most 0.1.
(0.501, 0.499, 0.5, 0.501, 0.499, 0.5)	Yes: each difference is at most 0.001.	The same guarantee applies, with changes in several directions.
First coordinate 0.502; the rest 0.5	No: one difference is 0.002.	This P ⇒ Q contract makes no promise about this pair.

The example with 0.5005 also shows that the premise is meaningful: two different inputs strictly inside the cube satisfy it. Lean checks such a witness explicitly. This is called **nonvacuity**. It prevents a statement from appearing successful merely because its input condition is impossible.

Where the bound comes from

A network combines weighted sums with activation functions. A weight scales a signal; a bias adds a constant. In these models the hidden activation is **ReLU**, which replaces a negative number by zero and leaves a nonnegative number unchanged. ReLU cannot enlarge the distance between two scalar values.

The checker follows how input differences can grow through the weight matrices. It uses absolute weights, so positive and negative contributions cannot cancel and hide a possible large difference. This gives a conservative bound: it may be larger than necessary, but the proof establishes that it is safe to use.

For DL2, Lean accepts a multiplier of **31**. For Gödel it accepts **26**. Consequently, a coordinate perturbation of at most 0.001 gives:

$$31 \cdot 0.001 = 0.031 \leq 0.1, \qquad 26 \cdot 0.001 = 0.026 \leq 0.1.$$

The proof is derived from the exact saved weights. It does not assume that training succeeded or infer success from a sample of predictions.

This is **robustness of the output to nearby inputs**. It does not by itself tell us whether the command is the right one for flying the drone. A constant network would also be insensitive to perturbations. The saved models here are nonconstant; evaluating control quality remains a separate task from bounding sensitivity.

3. Removing a neuron without changing the answer

A hidden neuron first computes a weighted sum plus a bias. Call this number its **preactivation**. ReLU then produces the neuron's activation. If we prove that the preactivation is never positive on the whole input domain, this neuron always contributes zero.

For **both DL2 and Gödel**, the first hidden layer's neuron 58, counting from zero, has the following checked upper bound:

$$z_{58}(x) \le \frac{-9057167}{134217728} < 0 \qquad (x \in [0,1]^6).$$

The upper bound is approximately −0.06748, so its activation is always zero. Whether an outgoing weight is 7, −4 or a very small fraction, its contribution is that weight multiplied by zero. Removing the neuron and its outgoing terms leaves the later weighted sums unchanged.

An **interval bound** encloses all possible values between a lower and an upper limit. The **pruning certificate** contains these bounds and a **mask**, a keep/delete flag for each neuron. A certificate is data to be checked, rather than a promise we accept on trust. Lean proves that if these checks pass, the deletion preserves the network's mathematical output throughout the cube:

$$\forall z \in [0,1]^6, \qquad N_{\text{small}}(z) = N_{\text{original}}(z).$$

For each of the two checkpoints	Before	After
Layer widths: inputs, hidden layers, output	6 → 64 → 32 → 1	6 → 63 → 32 → 1
Stored weights and biases	2,561	2,522

All six inputs remain. We remove a hidden computation, not an input coordinate. The saving is modest: 39 parameters, about 1.52%.

Why the smaller network inherits P ⇒ Q

1. **Prove the original promise**
   Every pair satisfying P has output difference at most 0.1.
2. **Prove exact reduction**
   The smaller network agrees with the original at every input in the cube.
3. **Substitute at both inputs**
   Replace N(x) and N(y) by the smaller network's equal outputs.

Therefore the smaller network satisfies the same implication, including the stronger 0.031 and 0.026 bounds. There is no new search over input pairs at this transfer step: equality transports the existing guarantee.

That answers the question for a fixed checkpoint. To reason about later changes, we must separate the evidence for equal outputs from the evidence that those outputs meet the requirement.

4. Two certificates answer different questions

The preceding result needs two independent pieces of evidence. One says that the original network satisfies the promise. The other says that making it smaller preserves its outputs.

Certificate	What it checks	What it allows us to conclude
Sensitivity certificate	All weight matrices and a proposed sensitivity bound.	Nearby inputs have sufficiently close outputs.
Pruning certificate	Hidden-layer bounds and the selected deletions.	The smaller and original networks have equal outputs on the domain.

This distinction becomes important when weights change. The pruning certificate does not inspect the output weights: zero hidden activations still contribute zero after those weights change. The sensitivity certificate does inspect them, because larger weights can amplify input differences.

A small arithmetic example

Consider an illustrative network with output $\mathbf{N(x) = x_0}$, plus an unused hidden neuron whose activation is always zero. A second version has output $\mathbf{M(x) = 200x_0}$, with the same unused neuron.

Take $x_0 = 0.5$ and $y_0 = 0.501$, keeping the other coordinates equal:

Network	Output difference	Meets the 0.1 requirement for this pair?
N	0.001	Yes
M	0.2	No

The zero neuron can be deleted in both cases. Deletion alone therefore cannot establish robustness after an arbitrary weight update. These two simple functions illustrate the logic; they are not the saved checkpoints.

Exact preservation also preserves a failure

Our earlier experiment used two other published checkpoints, called **Baseline** and **Adversarial**. They have smaller architectures than DL2 and Gödel. On these models, Lean establishes counterexamples to two directional requirements at margin 2. A **refutation** is an exact input where the premise holds but the requested output condition fails.

For example, a Baseline input strictly inside the cube has physical position sum $X + Y \approx 3.518069$. The selected requirement says that when $X + Y \geq 2$ the normalized output must be at least 0.5. Its output is approximately 0.399148. Lean checks the exact fractions behind these rounded figures. Its reduced network gives the same answer and hence the same counterexample. Appendix A gives the complete input.

Exact reduction faithfully preserves the behavior we have. Sections 10–11 return to this limitation and repair the Adversarial controller by changing its output function. First we follow the other consequence of reduction: what happens to information that no longer contributes to evaluation?

5. What are the state, the view and RR?

Removing a zero activation makes some weights irrelevant to the current answer. We now ask whether a representation that forgets those weights can still support future parameter changes. To keep every step visible, this part uses the **Adversarial model's certified hidden layers**, held fixed. It is a separate example from the positive DL2/Gödel property above.

Its second hidden layer has activations ($h_0(x)$, 0, $h_2(x)$, 0) on the cube. Here $h_0(x)$ and $h_2(x)$ are the surviving neurons' values, computed from x by the fixed hidden layers. The **output head** is the last layer: four weights and a bias, written (a_0, a_1, a_2, a_3, b). Thus its output is:

$$N_a(x) = \mathrm{ReLU}\,(a_0 h_0(x) + a_2 h_2(x) + b).$$

For this example a **state s** consists of an input x in the cube and that five-parameter head. The hidden layers are fixed background data. The **view q(s)** keeps x and the three parameters needed for evaluation:

$$s = (x, a_0, a_1, a_2, a_3, b), \qquad q(s) = (x, a_0, a_2, b).$$

Two different states, the same retained information

Fix any one input x in the cube. Consider these illustrative heads; the general Lean theorem allows every rational head (parameters that are ratios of integers), including these:

State	Full head (a_0, a_1, a_2, a_3, b)	Retained head (a_0, a_2, b)
s	(2, 7, 3, 11, 1)	(2, 3, 1)
t	(2, −4, 3, 99, 1)	(2, 3, 1)

Both evaluate as ReLU($2h_0(x) + 3h_2(x) + 1$). The different values 7 and −4 multiply zero; so do 11 and 99. The states differ, but the view cannot distinguish them.

RR names precisely this relationship:

$$RR(s, t) \iff q(s) = q(t).$$

RR is an **equivalence relation**: every state has its own view (reflexivity); equal views agree in either order (symmetry); and if two equalities share a middle view, the outer views agree (transitivity). Grouping states by their common view forms a **quotient**: each group retains one description of the information we chose to keep.

RR is stronger than “the outputs happened to match once.” It requires equality of the retained input and parameters. It also differs from P: **P compares nearby input vectors; RR compares full states with identical views.** This RR is defined on one state space, not between different types.

6. Can we still perform the next update?

Return to states s and t from the previous page. Both views contain the same head (2, 3, 1). Suppose we increase a_0 by 0.5. Both full heads then have $a_0 = 2.5$, and the reduced representation can perform exactly that update using information it already has.

Now try a different rule: **replace a_0 by $a_0 + a_1$**.

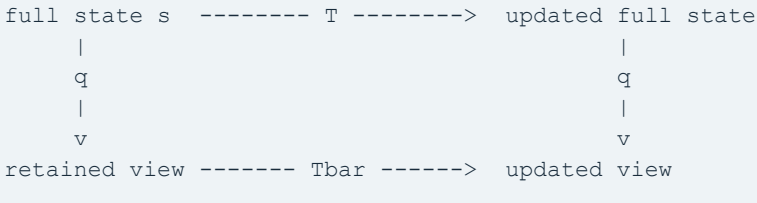
State	Old a_0	Forgotten a_1	New a_0
s	2	7	9
t	2	−4	−2

The reduced representation starts from (2, 3, 1) in both cases. It cannot deterministically choose both 9 and −2 as its new first coordinate. Information required by the update has been discarded. Lean proves this impossibility using the same pattern, with even smaller numeric witnesses.

The hidden neurons still contribute zero, so their pruning certificate remains valid. What fails is the ability to update using only the view.

The condition that makes an update compatible

Let **T** be an update on full states and **$\overline{T}$** its proposed reduced implementation. Compatibility means that these two routes agree:



$$q(T(s)) = \overline{T}(q(s)).$$

This equality is also called **factorization through the view**: the retained part of the result can be computed from retained information. If each permitted step has this property, **induction** gives it for every finite sequence: agreement holds before any steps; one compatible step preserves it; the argument repeats. Full and reduced execution stay in agreement. RR-related full states also remain RR-related under the same sequence. Choosing the next operation from the state must also use retained information.

An automatic dependency analysis can enlarge the view when necessary:

Permitted rules	Head coordinates that must be retained
Shared additive shifts	a_0, a_2, b: three coordinates
Also allow $a_0 := a_0 + a_1$	Add a_1: four coordinates
Also allow $a_1 := a_1 + a_3$	Add a_3: all five coordinates

Keeping an extra parameter for an update does not require restoring its zero hidden neuron. Parameter memory and inference computation serve different purposes.

7. Why does a subalgebra appear here?

The view tells us what information survives. We can also ask which quantities remain determined by that information. For the view q(s) = (x, a_0, a_2, b), consider:

Quantity we want to know	Can the view determine it?	Reason
a_0, or the sum $a_0 + a_2$	Yes	Both coordinates are retained.
The network output at the retained input	Yes	The deleted activations are zero.
a_1	No	The same view occurs with $a_1 = 7$ and $a_1 = -4$.
The value of a_0 after $a_0 := a_0 + a_1$	No	The rule reads information the view has forgotten.

A real-valued quantity attached to a state is called an **observable**. For example, F(s) = $a_0 + a_2$ is an observable. It factors through q because we can compute it from the view alone.

If two observables are determined by q, so are their sum and product, any constant multiple, and constant functions. That closure under arithmetic operations is why these observables form an **algebra**. They form a **subalgebra** of all real-valued functions on full states: some functions, such as the forgotten a_1, are excluded.

Formally, if V is the space of views, the subalgebra is:

$$\mathrm{Obs}(q) = \{f \circ q \mid f : V \longrightarrow \mathbb{R}\}.$$

Here composition means “first take the view, then apply f.” The condition is dependence only on the view; arbitrary real-valued functions are allowed, so this definition does not supply executable code for every member. Lean constructs an actual `Subalgebra` with this meaning.

A concrete calculation

For our shared view (x, 2, 3, 1), let F(s) = $a_0 + a_2$ and G(s) = b. We can recover **F = 5**, **G = 1**, **F + G = 6** and **FG = 5** from the view. Every state RR-related to s gives these same answers. We cannot recover a_1 by combining the retained quantities: s and t already show that it is not determined by them.

What “stable” means

After the compatible update $a_0 := a_0 + 0.5$, F becomes **5.5**. Its value changed, but it is still computable from retained information. A **stable subalgebra** keeps dependence on retained information after every allowed update. For any observable H in it, H after an update is still a function of the old view, via the reduced update.

That is the connection: **RR groups indistinguishable states; the subalgebra describes the quantities they agree on; compatible updates preserve our ability to calculate those quantities.** Stability does not say that outputs stay constant or that an earlier robustness bound automatically remains true.

For a learning rule, the next question is what else belongs in the state. The following examples show why the optimizer's memory matters too.

8. Learning adds memory to the state

An **optimizer** is a rule for changing parameters to reduce a loss, which is a numerical measure of error or another chosen objective. A **gradient** describes how the loss changes with each parameter. Many optimizers also keep memory of previous steps. A state sufficient for inference may therefore be insufficient for continued learning.

For **momentum**, let a be one parameter and v its stored update memory. With current gradient g, momentum factor μ and step size η, our convention is:

$$v' = \mu v + g, \qquad a' = a - \eta v'.$$

As a simple arithmetic illustration, set a = 2, g = 0.2, μ = 0.9 and η = 0.01. If v = 0, the next a is **1.998**. If v = 1, it is **1.989**. The same current parameter and gradient do not determine the next parameter unless the memory is also known.

Sometimes a sum is enough

The checked Adversarial-head experiment has five parameters and five momentum values: ten scalar coordinates. We specify an update with an extra loss term coupling a_0, a_1 and a_3 through their sum. This is a chosen experiment, not a reconstruction of the authors' training rule.

Individual coordinates would force us to keep all ten. A **linear observation** is a sum or weighted sum of coordinates. Such observations can instead retain these eight quantities:

$$q(s) = (a_0, a_1 + a_3, a_2, b,\ \ v_0, v_1 + v_3, v_2, v_b).$$

For example, (a_1, a_3) = (7, 11) and (8, 10) both give the retained sum **18**. Likewise, momentum pairs (4, 6) and (3, 7) both give **10**. With the other coordinates equal, the eight-value view cannot distinguish these full states, and the specified updates do not need to.

An exact calculation produces a candidate view and reduced update rule. Lean then checks that they agree with full execution. For the concrete coupled instance it also proves that eight is minimal among invariant linear observation spaces retaining the required output coordinates. Here **invariant** means the space remains sufficient after each allowed update: no new independent observation is needed to determine the next view. This is a precisely delimited minimum, not a claim about every possible encoding. With no coupling, the same comparison needs only six values.

The reduced rule must account for aggregation

If a_1 and a_3 each decrease by 1, their sum decreases by **2**. Updating their stored sum by only 1 would be wrong. The formal transport result includes this factor of two; a checked small example exposes the error when it is omitted. The benefit is an executable smaller state with a justified update rule, rather than merely fewer names on a page.

9. RMSProp shows why the allowed states matter

RMSProp, short for Root Mean Square Propagation, uses a running average of squared gradients to scale each parameter's step. A parameter with a different history can move differently even when its present gradient matches another's.

For a separate exact-real example, we use the following specified rule. The history r is nonnegative, the decay is 1/2, the step size is 1, and the stabilizing constant 1 is outside the square root:

$$r_i' = \frac{r_i + g_i^2}{2}, \qquad a_i' = a_i - \frac{g_i}{\sqrt{r_i'} + 1}.$$

Can we keep only the sum of two histories, as we kept sums for momentum? The full state now contains the five head parameters and five histories. The proposed view is (a_0, a_1+a_3, a_2, b, r_0, r_1+r_3, r_2, r_b): it keeps the head observations and replaces momentum by history observations. Start with the head (1, 0, 0, 0, 0). The specified coupled loss gives $g_1 = g_3 = 1$. All other histories are zero. Compare:

Initial histories (r_1, r_3)	Retained sum	Histories after the step	New weight sum $a_1' + a_3'$
(1, 97)	98	(1, 49)	−1/2 − 1/8 = −5/8
(49, 49)	98	(25, 25)	−1/6 − 1/6 = −1/3

The old view is identical, but the new weight sums differ. The separate square roots make the distribution of history matter. Lean proves that no deterministic update on this view works for **all nonnegative histories**.

A stronger assumption makes the same view sufficient

Suppose the paired histories start equal. In this model their gradients always agree, so their new histories also agree. This is an **invariant**: a condition true initially and preserved by every permitted step. Zero initialization satisfies it.

If $r_1 = r_3$ and their stored sum is R, each history must be R/2. Nothing needed by the square roots is missing. With $u = a_1 + a_3$ and their common gradient g, the exact reduced rule becomes:

$$R' = \frac{R + 2g^2}{2}, \qquad u' = u - \frac{2g}{\sqrt{R'/2} + 1}.$$

Lean proves this rule correct on the invariant equal-history states, including every finite permitted input sequence. RR and the stable subalgebra are defined on that restricted state space. The failing (1, 97) witness cannot occur from equal histories under these rules.

The sum fails on general histories but succeeds on the justified invariant states. This real RMSProp model has not yet been connected to the concrete rational-head checkpoints; the momentum example has that connection. We can now distinguish agreement through updates from satisfaction of a requirement. The next step addresses the latter: what if the behavior we have preserved needs to change?

10. From preserving an answer to repairing it

Return to the Adversarial checkpoint whose reduction preserved a directional failure. The task now is to produce different answers where the requirement demands them, while retaining useful predictions. Write d(x) for the physical position sum X + Y recovered from the six normalized inputs. The selected requirements are:

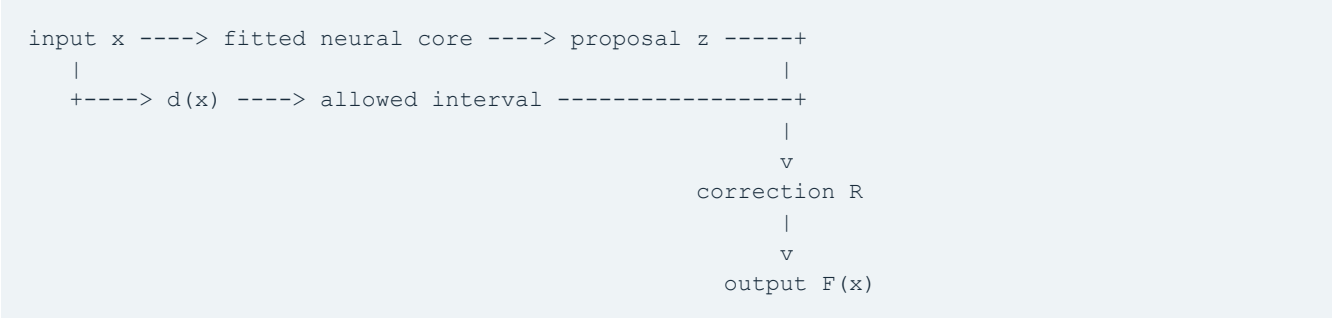
$$d(x) \geq 2 \Rightarrow F(x) \geq \tfrac{1}{2}, \qquad d(x) \leq -2 \Rightarrow F(x) \leq \tfrac{1}{2}.$$

They concern one input at a time. They are different from the nearby-input robustness requirement proved for DL2/Gödel. At the zero input, for example, d is about -41.19898449 and the original Adversarial output is about 1.36078430. That violates the second implication on the stated cube.

Let the core propose; let the block enforce the requirement

For each input, construct an interval of outputs that satisfy the selected directional requirement. When d is at least 2, the interval has lower endpoint 0.5. When d is at most -2, its upper endpoint is 0.5. Between the regions, the endpoints move continuously. Appendix E specifies them exactly.

The neural core proposes a scalar z. The correction operator R keeps z when it is inside the interval and otherwise returns the nearest endpoint. The complete controller therefore has two sources of information:



This is a composition that retains the input alongside the core's output:

$$H_\theta(x) = (x, N_\theta(x)), \qquad F_\theta = R_{64} \circ H_\theta, \qquad F_\theta(x) = R_{64}(x, N_\theta(x)).$$

The subscript 64 fixes the interval's transition slope. The proof permits every real proposal z, so the complete controller satisfies the two implications for any core parameters. For a concrete reference row with d about 2.070171, the fitted core proposes 0.369787 and the block returns 0.5. A proposal already inside the interval passes through unchanged.

11. Fit useful predictions, then certify reduction again

The block gives the directional guarantee, but those two implications alone would also allow a constant output of 0.5. We still need a useful fit to the supplied reference commands. Training therefore adjusts the core through the fixed block: the loss measures the complete controller's output against each original label.

$$\mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(R_{64}(x^{(i)}, N_\theta(x^{(i)})) - y_i\right)^2, \qquad m = 360.$$

Here each x with a parenthesized superscript is one input vector and y_i is its stored target. The optimizer proposes parameters; they are then frozen as exact rational values. Lean checks the resulting reference error and the universal directional guarantees.

Controller on the original 360 labels	Mean squared error
Published Adversarial checkpoint	0.001532418405
Frozen fitted core plus correction	0.001475236171
Constant output of 0.5	0.109078851968

The repaired result has about 3.73% lower error on these rows, all of which were also used for fitting. This is a reference-quality check, not a held-out result. Labels are not rewritten to make the repair look better: the preceding example's label is about 0.373198, so enforcing 0.5 incurs error there. Better agreement elsewhere makes the complete fit pass.

Equality brings reduction back into the construction

The fitted core changes hidden weights as well as the output head. We therefore check a new pruning certificate. It proves that the fitted full and small cores agree throughout the cube. Composing each with the same correction block preserves that equality:

$$N_{\text{small}}(x) = N_{\text{full}}(x) \quad \Longrightarrow \quad R_{64}(x, N_{\text{small}}(x)) = R_{64}(x, N_{\text{full}}(x)).$$

Thus the smaller repaired controller keeps both the directional guarantee and the reference predictions of the full repaired version. Its core has 41 parameters instead of 75. The added block uses six ReLU operations; the complete small repaired graph has 13, compared with 17 for the full repaired graph and 11 for the old unrepaired network.

The fitted core supplies the predictions, the block enforces the selected requirement, and equality justifies the reduction. Appendix E follows every formula to its code and checked theorem.

12. How preservation, stable views and repair fit together

The original research connects logical requirements with machine learning. The examples here follow three questions that arise when putting that connection to work. First, can a checked property survive removal of computation? Second, can the retained representation still support future updates? Third, can a changed controller satisfy a requirement that its predecessor violated?

Question	Mathematical relation	Checked role
Can we remove computation?	Equal outputs on the domain.	A smaller network inherits the full network's output properties.
Can we forget information?	The next view is determined by the current view.	RR and a stable observable subalgebra justify the specified reduced updates.
Can we restore a requirement?	The corrected output belongs to an allowed interval.	Core plus block satisfies both directional implications; exact computation checks reference quality.

Composition provides the concrete connection to the repair. If two cores give the same proposal at the same input, the common block gives the same corrected output. The proof that correction enforces the requirement and the proof that reduction preserves the proposal can therefore be used together.

There is also a precise information question for any future integration with the subalgebra. The correction reads d(x) as well as the proposed output. Retaining enough information to determine both is sufficient to execute this construction from a view. The earlier view retains x, but its update proofs assumed fixed hidden layers. The present fit changes those layers, so that old certificate does not automatically cover this model.

The established cases remain identifiable: DL2/Gödel provide the positive robustness-and-reduction result; Adversarial supplies the fixed-hidden-layer view examples and, separately, the new directional repair. The RMSProp example has its own real state space. Their roles fit together without asserting a single theorem about all these systems at once.

A concrete question for the next collaboration

Which additional controller requirement should remain true while the repaired core is adapted? With the same proved block, the selected directional implications already hold independently of the core weights. Preserving useful reference behavior, other output properties and an executable reduced update rule would require their own acceptance checks or invariants. That is where the repair and stable-view studies could meet in a further verified experiment.

Before choosing that experiment, we should distinguish the mathematical network from its numerical execution and from the physical system it would control. The next section makes those boundaries explicit.

13. What the proof says about a running program

The fixed-checkpoint proofs interpret saved weights exactly and perform network arithmetic over the real numbers. Each finite float32 checkpoint constant has an exact rational value; that value is retained in the model. The repaired core instead uses its newly frozen rational parameters. A **rational** is a ratio of integers. Inputs range over reals, so the universal statements are not restricted to finitely many representable inputs.

Actual float32 computation rounds intermediate results. Changing matrix shape can change the order of additions and hence the rounding. Exact real equality alone does not imply identical floating-point execution.

An observed difference in the last bit

In an earlier Baseline comparison, one input produced:

```
original output: 0.9654448628425598
reduced output:  0.965444803237915
```

These adjacent float32 values illustrate the distinction. Across 8,616 tested inputs, Baseline PyTorch batch-one execution differed on 337; the largest difference was approximately 0.000000477. Other tested execution layouts were bit-identical on that corpus. The real theorem does not claim to settle these runtime differences.

For DL2 and Gödel, original and reduced ONNX models were bit-identical on **4,520 inputs per model** in both tested layouts: single-input and whole-corpus execution. That corpus contains 360 published data rows, 64 cube corners and 2,048 pairs of nearby inputs. These observations support the implementation check; a universal float32 theorem would require a further numerical argument.

What has and has not been measured

The saved DL2/Gödel models were not retrained for their reduction and robustness checks. Their predictions are nonconstant. On the 360 published rows, their descriptive mean squared errors (the average squared difference from the stored targets) were approximately 0.025110 for DL2 and 0.033898 for Gödel, compared with 0.081213 for the best constant on those same rows. It is unknown which rows were held aside from these checkpoints' training, so these figures do not establish performance on unseen data or acceptable flight performance.

The separate Adversarial repair does fit a new core, as Section 11 explains. Its quality check uses the same 360 rows as fitting. Its exact-real directional proof does not establish a universal float32 guarantee or a closed-loop flight result.

The parameter counts describe structural savings. They do not by themselves promise lower latency on a particular processor. A controller's flight behavior also depends on sensing, normalization, actuator dynamics and the physical system, which are outside the network theorem.

This separates three kinds of evidence a collaborator can examine: **mathematical guarantees**, **finite execution checks**, and **the remaining application evaluation**. The appendices locate each artifact without requiring the reader to work through every proof file first.

Appendix A. Exact models and reproducible examples

All four source checkpoints come from the authors' Alsomitra artifact repository, at revision `e9f6f1c52e0e5349455d2b4f69c2015567d3037f`. DL2/Gödel are under `vehicle_verification/property_verification_2/`; Baseline/Adversarial are under `python_training/models/`.

Checkpoint	Original → reduced widths	Parameters	Output activation
DL2 / Gödel, each	6-64-32-1 → 6-63-32-1	2,561 → 2,522	Linear
Baseline	6-6-4-1 → 6-5-4-1	75 → 64	ReLU
Adversarial	6-6-4-1 → 6-4-2-1	75 → 41	ReLU

The larger models have two hidden ReLUs and no final ReLU. The smaller pair has a final ReLU as well. The preservation proofs respect that difference. The original files remain unchanged; reduced exports are reloaded and their parameter bits bound to the formal reduced networks.

The exact bound behind the positive property

For weight matrices W_1, W_2, W_3, propagate the vector of initial unit bounds through their elementwise absolute values:

$$c_0 = (1, 1, 1, 1, 1, 1), \quad c_1 = |W_1|c_0, \quad c_2 = |W_2|c_1, \quad c_3 = |W_3|c_2.$$

The scalar result is approximately 30.332955 for DL2 and 25.385532 for Gödel; the exact checks accept 31 and 26. This is a standard conservative method, not an optimality claim. Biases cancel when comparing outputs of the same affine layer at two inputs.

The companion paper's Section 7.2 and Vehicle `LipschitzRobustness` use an output threshold Lstar/epsilon. The selected published pair is epsilon = 0.001 and Lstar = 0.0001, giving 0.1. Lstar is **not** the proved Lipschitz multiplier 31 or 26. The paper's Table 2 evaluates different model variants; applying its threshold here does not reproduce one of its reported cells. The Vehicle-to-Lean correspondence is reviewed manually; no verified Vehicle compiler is claimed.

The directional counterexample from Section 4

Input order is [vx, vy, omega, theta, x, y]. Take the Baseline input [1/4, 1/2, 1/2, 1/4, 1/4, 3/4]. The physical position conversions are:

```
X = 41.232296222734604 * input[4] + 0.482420778264618
Y = 45.878638794593570 * input[5] - 41.681405266088724
```

Lean proves X + Y > 2 and output < 1/2 using exact arithmetic. The property-decision guide gives all four original-model witnesses and their transfers. These refutations are consistent with the companion paper's larger reported critical distances; they are not a claim of a newly discovered flight failure.

Appendix B. Details behind the update examples

Certificates, state and preservation

The pruning checker inspects intervals and keep masks. For a neuron z = sum($w_i x_i$) + b in an input box, it checks an upper bound obtained by summing each term's maximum. Propagating those bounds through ReLU encloses the next layer. A deleted neuron must have upper bound at most zero. This may miss removable neurons when it loses correlations, but an accepted deletion is sound.

The head-update example's state type includes the evaluation input. The momentum and RMSProp modules instead take evaluation features as external arguments. Their states contain parameters and optimizer memory. In each case q is defined on that stated space; RR always means equality of its views.

The specified coupled momentum rule

Let θ = (a_0,a_1,a_2,a_3,b), let z = (h_0,0,h_2,0,1) be frozen features, and let y be a target output. The chosen loss is:

$$L(\theta) = \frac{(\theta \cdot z - y)^2}{2} + \frac{\lambda}{2}(a_0 + a_1 + a_3)^2.$$

Its algebraically specified update direction is G = (θ·z − y)z + λ(a_0+a_1+a_3)(1,1,0,1,0). The steps are v′ = μv + G and θ′ = θ − ηv′. No automatic differentiation implementation is verified.

The concrete synthesis instance uses checkpoint-derived features from the first published training row, y = 3/4, η = 1/100, μ = 9/10 and λ = 1/100. A second instance has λ = 0. The general correspondence proof allows rational configurations to vary across a finite sequence. It compares both networks at the resulting current heads; it makes no claim that the original output guarantee or loss value stays unchanged.

An **affine update** multiplies a state by a matrix and adds a constant vector. For T(s) = As + c, a linear view q(s) = Bs is checked by BA = ĀB and Bc = c̄. These equations justify the reduced rule v ↦ Āv + c̄. Repeatedly adding the dependencies of required observations produces a closed row space. Each matrix row specifies one weighted sum; its row space contains all linear combinations of those sums. Closure means the updates require no observation outside that space. Lean checks the emitted derivations and the scoped eight-dimensional minimum.

For a head projection B and a direction of the form G(θ) = B^Tg(Bθ), the projected direction is BB^Tg, not generally just g. In this example BB^T = diag(1,2,1,1): the sum coordinate accounts for two full coordinates. This supplies the factor of two explained in Section 8.

The RMSProp invariant

The separate real model uses features (0,0,z_2,0,1), target zero and coupled penalty (a_0+a_1+a_3)²/2. Its paired gradients always agree. Consequently r_1' − r_3' = (r_1 − r_3)/2; equal nonnegative histories remain equal and nonnegative. The reduced rule is proved on this subtype, for finite sequences of real z_2. Square roots can yield irrational parameters, so this development does not silently reuse the concrete rational-head evaluation theorem.

Appendix C. Where the claims are checked

Lean's **kernel** checks that a proof establishes its stated theorem. The theorem's definitions and assumptions still determine its meaning. The local checkers reject invalid certificates; the Python programs that propose bounds or reductions are not trusted to establish a theorem.

Claim in this guide	File and named result
Exact P/Q meaning and nonempty premise	Lipschitz.lean: `Property5`, `property5_two_sided`, `nonvacuous`.
Accepted sensitivity bound implies the property	Same file: `checked_global`, `property5_of_check`.
Positive checkpoint and reduced-model implications	DL2Checkpoint, GodelCheckpoint: `property5`, `same_output`, `reduced_property5`.
Exact pruning with a linear output	LinearPruning: `certified_linear_prune`.
RR, compatible updates, subalgebra and failure	OutputUpdates: `stateSetoid`, `run_factors`, `view_algebra_stable`, `bad_no_reduced_step`.
Momentum correspondence and retained-state synthesis	MomentumQuotient, LinearObservations; guide.
RMSProp failure and invariant-based success	AdaptiveBoundary, SymmetricRMSProp.
Property and counterexample transfer	PropertyTransfer, Witnesses.

The previously recorded audits cover the following roots. Counts overlap; they should not be added to claim a number of distinct new theorems.

Recorded development	Audit roots and verification receipt
Checkpoint pruning	43 roots
Output-head updates	36 roots
Coordinate retention	78 roots
Linear observations	82 roots
RMSProp / property transfer	47 roots
Concrete property decisions	48 roots
Positive property and its reduction	53 roots

The accepted audits report only Lean's standard `propext`, `Classical.choice` and `Quot.sound` axioms. The positive-property run checks nine compilation stages with no errors or warnings. Negative controls reject a zero sensitivity bound, deletion of active neurons and a wrong ONNX transpose. There is no new second-kernel validation. For this shared snapshot, the same proof sources are rebuilt in an isolated checkout; the linked receipts record that replay. The mathematical statements are unchanged.

Appendix D. Try the positive example first

The shortest path through the artifacts is the positive property on DL2/Gödel. Read its focused guide, then run the following command from this repository's root:

```
python -B contribution/network_reduction/positive_property/verify.py \
  --upstream /path/to/SAIV_2025_Alsomitra
```

Replace the path with a checkout at the revision in Appendix A. Use a Python environment with NumPy, ONNX and ONNX Runtime. The runner uses the repository's pinned Lean/mathlib installation and validates the six earlier PASS receipts and their proof-source bindings. It rebuilds the needed local proof modules, regenerates both reduced ONNX exports and runs the finite execution comparisons. The focused guide documents setup and artifact locations; these commands are not a standalone installation of Lean or the runtime packages.

The resulting positive-property receipt records the source hashes, compilation results, checked theorem roots, invalid-certificate controls and finite runtime evidence. A PASS means those checks succeeded. For the separate refutation experiment, PASS means the counterexample was verified, not that the refuted controller property was true.

For other parts, the guides provide the exact state definitions and runners:

- Fixed-checkpoint reduction and head updates.
- Coordinate retention and linear observations.
- Equal-history RMSProp and property transfer.
- Directional property decisions.

The synthesis stages validate predecessor bindings. When replaying their full chain after a document revision, regenerate predecessors in order rather than treating an expected old-document hash mismatch as a failed mathematical theorem. The repository README retains the ordered commands.

Sources and relation to the original work

- A General Framework for Property-Driven Machine Learning, Thomas Flinkow et al., 2025, version 2: the originating methodology.
- Neural Network Verification for Gliding Drone Control: A Case Study, Colin Kessler et al., 2025: controller requirements and companion experiments, particularly Section 7.2.
- Pinned Alsomitra repository: the exact model and specification artifacts used here.

The training runs that produced the original upstream checkpoints are not reproduced. Appendix E describes a separate repair fit. Exact reduction, linear observation spaces and equality-based property transfer are existing mathematical ideas; no priority claim is made for them. The inspectable result here is their checked application to the stated artifacts and update rules, with examples showing both where the reasoning succeeds and where an extra assumption is necessary.

To rebuild this PDF, install ReportLab, markdown-it-py, Matplotlib and svglib in the document environment and use the macOS fonts named by the shared renderer, then run:

```
python contribution/network_reduction/build_report.py
```

Appendix E. Mathematical workflow of controller repair

This appendix explains the complete method behind the worked repair example introduced in Sections 10–12. It expands the composition step by step, from allowed outputs to the fitted and certified smaller controller. Experiments helped select the construction and its parameters. Its universal guarantee comes from a proof about the correction operator; its reference-quality claim comes from an exact check of the frozen candidate on the supplied data.

The starting object is the published **Adversarial** controller. The result is a fitted neural core followed by an explicit correction block. The architecture changes. The two requirements here are **Above(2)** and **Below(2)**; the earlier two-input robustness theorem is not being asserted for this repaired controller.

```
Specify the domain and requirements
                 |
                 v
Define the allowed outputs for each input
                 |
                 v
Construct and prove a correction operator
                 |
                 v
Fit the neural core through that operator
                 |
                 v
Freeze exact parameters and check reference quality
                 |
                 v
Certify reduction and transfer the output guarantees
```

E.1. Inputs, physical coordinates and the contract

Let the six normalized input coordinates be horizontal velocity, vertical velocity, angular velocity, angle, horizontal position and vertical position, in that order. A neural core maps each permitted input to one real output; theta denotes its weights and biases.

$$D = [0, 1]^6, \qquad x = (x_0, x_1, x_2, x_3, x_4, x_5), \qquad N_\theta : D \longrightarrow \mathbb{R}.$$

The physical coordinates and displacement use the existing normalization constants. These decimal literals denote exact rational numbers in the formal model.

$$X(x) = 41.232296222734604\, x_4 + 0.482420778264618.$$

$$Y(x) = 45.878638794593570\, x_5 - 41.681405266088724, \qquad d(x) = X(x) + Y(x).$$

E.2. From implications to allowed output sets

The input predicates select two regions. The output predicates specify the permitted side of one half. The letter z denotes a scalar output.

$$P_+(x) \iff d(x) \geq 2, \qquad Q_+(z) \iff z \geq \tfrac{1}{2}.$$

$$P_-(x) \iff d(x) \leq -2, \qquad Q_-(z) \iff z \leq \tfrac{1}{2}.$$

The name Correct below means exactly the conjunction of these two directional requirements. It does not mean correctness of every aspect of the physical controller.

$$\text{Correct}(N) \iff \forall x \in D, \quad (P_+(x) \Rightarrow Q_+(N(x))) \wedge (P_-(x) \Rightarrow Q_-(N(x))).$$

Equivalently, define the allowed output set separately in each region:

$$d(x) \geq 2 \quad \Longrightarrow \quad A(x) = [\tfrac{1}{2}, +\infty).$$

$$d(x) \leq -2 \quad \Longrightarrow \quad A(x) = (-\infty, \tfrac{1}{2}].$$

$$-2 < d(x) < 2 \quad \Longrightarrow \quad A(x) = \mathbb{R}.$$

The regions are disjoint and cover the domain, so this defines A uniquely. The same contract now reads:

$$\text{Correct}(N) \iff \forall x \in D, \quad N(x) \in A(x).$$

A constant output of one half satisfies both implications. A separate quality gate will exclude that uninformative solution on the reference data. There is no directional requirement in the open gap between -2 and 2.

E.3. A nonempty interval inside the allowed set

Choose a fixed nonnegative slope k. The published candidate uses k = 64. Define lower and upper bounds, then an input-dependent interval:

$$L_k(d) = \min\left(\tfrac{1}{2}, \max\left(0, \tfrac{1}{2} + k(d-2)\right)\right).$$

$$U_k(d) = \max\left(\tfrac{1}{2}, \min\left(1, \tfrac{1}{2} + k(d+2)\right)\right), \qquad I_k(x) = [L_k(d(x)), U_k(d(x))].$$

The interval bounds and their values in the two required regions give:

$$0 \le L_k(d) \le \tfrac{1}{2} \le U_k(d) \le 1.$$

$$k \ge 0: \qquad d \ge 2 \Rightarrow L_k(d) = \tfrac{1}{2}, \qquad d \le -2 \Rightarrow U_k(d) = \tfrac{1}{2}.$$

Consequently the interval is always nonempty and lies inside the allowed set. It additionally restricts outputs to [0,1] and narrows part of the otherwise unconstrained gap. It describes a sufficient family of solutions, not every controller satisfying the original contract.

$$\forall x \in D, \qquad I_k(x) \neq \varnothing \quad \wedge \quad I_k(x) \subseteq A(x).$$

E.4. The correction operator and its laws

Project the neural core's proposed output z into that interval:

$$R_k(x, z) = \max\left(L_k(d(x)), \min\left(U_k(d(x)), z\right)\right).$$

Below the interval, return its lower endpoint; inside it, keep z; above it, return its upper endpoint. On reference row 129, for example, d is about 2.070171 and the fitted core proposes about 0.369787. The corrected output is 0.5, as required. The original label is about 0.373198 and remains unchanged when quality is evaluated.

The operator has three checked laws: membership, preservation of an already admissible proposal, and idempotence. Idempotence means a second application at the same input does not change the result again.

$$\forall x \in D,\ \forall z \in \mathbb{R}, \qquad R_k(x, z) \in I_k(x).$$

$$z \in I_k(x) \quad \Longrightarrow \quad R_k(x, z) = z.$$

$$R_k(x, R_k(x, z)) = R_k(x, z).$$

The central inference is simply membership followed by set inclusion:

$$\frac{R_k(x, z) \in I_k(x) \qquad I_k(x) \subseteq A(x)}{R_k(x, z) \in A(x)}.$$

This mathematical operator has an explicit neural implementation, using:

$$\max(a, b) = a + \mathrm{ReLU}(b - a), \qquad \min(a, b) = a - \mathrm{ReLU}(a - b).$$

Sharing the computed endpoints gives six additional ReLU operations. `reluBlock_eq_project` proves that this expression equals the projection. The guarantees belong to the complete construction, including this block.

E.5. Composition gives a family of correct controllers

For fixed k at least zero, put the operator after an arbitrary neural core:

$$F_\theta(x) = R_k(x, N_\theta(x)).$$

Because the operator's proof allows every real proposal z, it follows that:

$$\forall \theta,\ \forall x \in D, \quad F_\theta(x) \in A(x), \qquad \text{hence} \quad \forall \theta,\ \mathrm{Correct}(F_\theta).$$

These are the stated exact-real directional guarantees. Changing the core parameters leaves them intact while the same correct block and input interpretation remain in place. Reference quality and pruning obligations must still be checked for a changed core. No global sensitivity or flight stability result follows from this membership argument.

E.6. Fitting chooses useful predictions within that family

The reference data form an indexed collection of input-label pairs; all 360 rows are used for fitting and for the later quality check. A row's label is its reference output, not a new constraint on every possible input.

$$T = ((x^{(i)}, y_i))_{i=1}^{m}, \qquad m = 360.$$

The parenthesized superscript distinguishes the i-th input vector from a coordinate such as x_4. The fitting objective is mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(F_\theta(x^{(i)}) - y_i\right)^2.$$

Substituting the composed controller makes the role of the block explicit:

$$\mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(R_{64}(x^{(i)}, N_\theta(x^{(i)})) - y_i\right)^2.$$

The optimizer fits through this fixed correction block. It produces a candidate parameter vector; there is no proof of a global minimum or a minimal change from the original weights. The training procedure and replay describe how this particular candidate was found. The operator proof does not depend on the optimizer finding it.

E.7. Freeze the candidate before checking its quality

The floating training result is converted to dyadic rational weights by rounding to the nearest multiple of 1/65536, with ties to even. The index j selects a weight or bias. The formal result concerns this frozen object:

$$\bar{\theta}_j = \frac{\mathrm{round}(65536\,\theta_{\mathrm{candidate},j})}{65536}, \qquad \bar{F}(x) = R_{64}(x, N_{\bar{\theta}}(x)).$$

For any output function G, define the finite reference error:

$$\mathrm{MSE}_T(G) = \frac{1}{m} \sum_{i=1}^{m} \left(G(x^{(i)}) - y_i\right)^2.$$

Let N_old be the original published Adversarial controller, and let C always return one half. The quality gate is:

$$C(x) = \tfrac{1}{2}, \qquad \mathrm{Quality}_T(G) \iff \mathrm{MSE}_T(G) \le \mathrm{MSE}_T(N_{\mathrm{old}}) \ \wedge \ \mathrm{MSE}_T(G) < \mathrm{MSE}_T(C).$$

Exact kernel-checked computations, transferred to the real evaluation semantics, establish the following bounds for the frozen candidate:

$$\mathrm{MSE}_T(\bar{F}) \le \frac{1476}{1000000} = 0.001476.$$

$$\mathrm{MSE}_T(N_{\mathrm{old}}) \ge \frac{1532}{1000000} = 0.001532, \qquad \mathrm{MSE}_T(C) \ge \frac{109}{1000} = 0.109.$$

The inequalities have a strict separation, so they prove the gate:

$$\mathrm{MSE}_T(\bar{F}) \le 0.001476 < 0.001532 \le \mathrm{MSE}_T(N_{\mathrm{old}}).$$

$$\mathrm{MSE}_T(\bar{F}) \le 0.001476 < 0.109 \le \mathrm{MSE}_T(C).$$

The measured MSEs are approximately 0.001475236171 for the repair and 0.001532418405 for the original, a reduction of about 3.73% on these rows. The finite quality claim is itself checked mathematically; its scope is the reference collection, not an unseen population. It is not a held-out evaluation. The original labels remain unchanged, including ten that conflict with Above(2).

E.8. Reduction is justified by functional equality

The fitted small core is embedded into a full core. A new interval certificate establishes that the four omitted neurons remain inactive throughout D. With the model's dimension and domain conditions, the general pruning theorem and that accepted certificate give:

$$\forall x \in D, \qquad N_{\mathrm{small}}(x) = N_{\mathrm{full}}(x).$$

Here N_small is the frozen fitted core. N_full is its full embedded version; neither name denotes the unchanged original checkpoint.

Apply the same correction block to both cores:

$$F_{\text{small}}(x) = R_{64}(x, N_{\text{small}}(x)), \qquad F_{\text{full}}(x) = R_{64}(x, N_{\text{full}}(x)).$$

Equality transfers through this common operation:

$$\frac{N_{\text{small}}(x) = N_{\text{full}}(x)}{R_{64}(x, N_{\text{small}}(x)) = R_{64}(x, N_{\text{full}}(x))}.$$

Consequently the two repaired versions agree for every input in D and have identical error on reference rows in D. Output properties transfer along this equality. Repair intentionally changes the original function.

```
Fitted full core:   6 -> 6 -> 4 -> 1 --+--> same block --> F_full
                                       |                    ||
Certified equality throughout D ------+                    ||
                                       |                    ||
Fitted small core:  6 -> 4 -> 2 -> 1 --+--> same block --> F_small
```

The core parameter count goes from 75 to 41. Including the six added ReLUs, the full repaired graph uses 17 ReLU operations and the smaller one 13. The original unrepaired graph used 11, so this comparison does not claim that the repaired graph is smaller than the original unrepaired network.

E.9. The combined result and its scope

Writing F_small for the frozen small core plus block, the final theorem establishes all four of these clauses together:

$$\forall x \in D, \qquad d(x) \geq 2 \implies F_{\text{small}}(x) \geq \tfrac{1}{2}.$$

$$\forall x \in D, \qquad d(x) \leq -2 \implies F_{\text{small}}(x) \leq \tfrac{1}{2}.$$

$$\text{MSE}_T(F_{\text{small}}) \leq \text{MSE}_T(N_{\text{old}}) \quad \wedge \quad \text{MSE}_T(F_{\text{small}}) < \text{MSE}_T(C).$$

E.10. Where each step meets code and proof

The set notation A, I and Correct above is explanatory notation for the concrete definitions and theorems below. It does not denote additional Lean declarations or a newly verified general-purpose repair framework.

Mathematical step	Implementation and checked statement
Allowed interval and operator laws	ProjectionRepair.lean: `interval_valid`, `project_in_interval`, `project_above`, `project_below`, `project_unchanged`, `project_idempotent`.
Six-ReLU expression	Same file: `reluBlock_eq_project`; RepairNetwork.lean: `repaired_relu_program`.
Physical inputs and arbitrary-core guarantees	RepairNetwork.lean: `displacement_physical`, `repaired_above`, `repaired_below`.
Candidate selection and freezing	train.py, freeze.py, CANDIDATE.json. Training proposes the parameters; freezing fixes the exact object.
Exact reference data and quality	RepairReference.lean: `reference_count`, `reference_in_cube` and the three MSE bounds; RepairResult.lean: `reference_quality`.
Rational computation and real semantics	RepairNetwork.lean: `repaired_cast`, `mse_cast`.
Certified reduction of the fitted model	RepairedCore.lean: `pruning_checked`, `core_equal`; RepairNetwork.lean: `smaller_repair_equal`.
Combined contract	RepairResult.lean: `repaired_controller_contract`.

The recorded repair verification audits 39 theorem roots, reports no errors or warnings, and permits only the three standard axioms listed in Appendix C. This appendix explains those existing results; its addition is not a new proof run. Reproduction commands and exact runtime examples are in the repair walkthrough.

The universal clauses concern real input values and the exact rational parameters interpreted over the reals. The MSE clauses concern the fixed 360-row collection. Float32 execution, physical reachability of every point in the cube, and closed-loop flight behavior require separate arguments. The earlier observable-subalgebra and update certificates are not automatically extended to this changed architecture.